FEASIFY BUG & DEFECT REGISTRY

Document ID: FEASIFY-DOC-010
Version: 1.0.0
Date: November 10, 2026
Author: Lead QA Orchestrator & Multi-Agent Defect Triage
Classification: Formal Defect Tracking & Remediation Guide

---

1. Defect Summary & Severity Distribution

SEVERITY	COUNT	PRIORITY	DEFINITION
Critical	2	P1	Immediate security compromise, severe data exposure, or complete authorization failure.
High	4	P2	Core business rule bypass, unhandled fatal runtime crash, or API resource exhaustion risk.
Medium	4	P3	Partial feature impairment, configuration weakness, or linting failure.
Low	3	P4	Minor formatting flaw, bundle size warning, or benign typing mismatch.
Total Defects	**13**	—	—

---

2. Detailed Bug Registry

FEASIFY-BUG-001: Broken ESLint Flat Config Crashes Linting Suite

- **Module:** Code Quality / Tooling
- **Severity: Medium** | **Priority: P3**
- **Affected File:** `Feasify/eslint.config.cjs` (line 11)
- **Preconditions:** Node.js v22 installed with eslint-plugin-react-hooks v5.
- **Steps to Reproduce:** Run `npm run lint --prefix Feasify`.
- **Expected Result:** ESLint parses TypeScript/React codebase and reports linting findings.
- **Actual Result:** `TypeError: Cannot read properties of undefined (reading 'recommended') at eslint.config.cjs:11:27`.
- **Root Cause:** In `eslint-plugin-react-hooks` v5, the configuration object structure changed and does not contain `flat.recommended`.
- **Recommended Solution:** Replace `reactHooks.configs.flat.recommended` with `reactHooks.configs.recommended`.
- **Retest Status:** Open (Testing stage; no code modifications permitted without approval).

FEASIFY-BUG-002: Fatal ReferenceError in Student Settings Profile

- **Module:** Student Profile / Settings
- **Severity: High** | **Priority: P2**
- **Affected File:** `Feasify/src/Settings.tsx` (lines 540, 559)
- **Preconditions:** Authenticated Student opens `/settings` or `/profile`.
- **Steps to Reproduce:**
  1. Open user profile tab in Settings.
  2. Input a new username handle.
  3. Click "Save Username".
- **Expected Result:** Username is saved to Firestore; success toast appears; modal closes.
- **Actual Result:** Runtime crash: `Uncaught ReferenceError: newUsername is not defined` and `setShowUsernameModal is not defined`.
- **Root Cause:** Identifiers `newUsername` and `setShowUsernameModal` are referenced in `handleChangeUsername` without being declared in component state.
- **Recommended Solution:** Declare `const [newUsername, setNewUsername] = useState("");` and `const [showUsernameModal, setShowUsernameModal] = useState(false);` in `Settings.tsx`.
- **Retest Status:** Open.

FEASIFY-BUG-003: Direct Client Firestore Calls Bypass 3-Proposal Quota

- **Module:** Proposal Management / Security
- **Severity: High** | **Priority: P2**
- **Affected File:** `Feasify/src/Projects.tsx` (line 1787)
- **Preconditions:** Student has reached the maximum of 3 proposals.
- **Steps to Reproduce:** Open browser developer console and execute `addDoc(collection(db, "proposals"), newProposalPayload)`.
- **Expected Result:** Firestore rejects write with permission denied due to quota limits.
- **Actual Result:** Firestore accepts write, creating a 4th proposal for the group.
- **Root Cause:** Proposal quota is verified in frontend React state and a backend query, but Firestore lacks a server-side Security Rule / Cloud Function to enforce group quota atomically.
- **Recommended Solution:** Route all proposal submissions through backend endpoint `POST /api/teams/:groupId/proposals` or implement a Firestore transaction counter.
- **Retest Status:** Open.

FEASIFY-BUG-004: Incomplete Whitespace Normalization in Sandbox DTI Result

- **Module:** Name Checker Service
- **Severity: Low** | **Priority: P4**
- **Affected File:** `feasify-backend/services/nameChecker/dti/DTIProvider.js` (line 224)

- **Preconditions:** DTI sandbox mode active.
- **Steps to Reproduce:** Query `" cocopan bakery "`.
- **Expected Result:** `buildResult` stores normalized query with collapsed spaces `"cocopan bakery"`.
- **Actual Result:** `buildResult` stores trimmed query with internal multiple spaces intact (`"cocopan bakery"`).
- **Root Cause:** `_handleSandboxQuery` passes un-collapsed `query` parameter into `buildResult({ query, ... })`.
- **Recommended Solution:** Pass `this.normalize(query)` into `buildResult`.
- **Retest Status:** Open.

FEASIFY-BUG-005: Missing Authentication on Proposal Status Mutation API

- **Module:** Companion Backend / Authorization
- **Severity: Critical** | **Priority: P1**
- **Affected File:** `feasify-backend/index.js` (lines 773-807)
- **Preconditions:** Backend running on port 10000.
- **Steps to Reproduce:** Send `PUT /api/proposals/:proposalId/status` with `{"status": "Approved"}` without any `Authorization` header.
- **Expected Result:** HTTP `401 Unauthorized` or `403 Forbidden`.
- **Actual Result:** HTTP `200 OK`; proposal updated in Firestore using Firebase Admin SDK.
- **Root Cause:** Route handler was defined without `verifyToken` middleware.
- **Recommended Solution:** Add `verifyToken` middleware and verify `req.user.role === "Adviser" || req.user.isChairperson`.
- **Retest Status:** Open.

FEASIFY-BUG-006: Plaintext Password Storage in Firestore Document Store

- **Module:** Chairperson Administration / Authentication
- **Severity: Critical** | **Priority: P1**
- **Affected File:** `Feasify/src/ChairpersonModule.tsx` (lines 253, 484)
- **Preconditions:** Chairperson creates user or imports roster via Excel.
- **Steps to Reproduce:** View the Firestore `users` collection in Firebase Console or query via client SDK.
- **Expected Result:** Passwords exist solely inside Firebase Auth credential store.
- **Actual Result:** Plaintext string attribute `password: "DELACRUZ-1234"` stored in Firestore document.
- **Root Cause:** Chairperson component copies the auto-generated temporary password into the Firestore user document object.
- **Recommended Solution:** Delete `password` property from `userData` object before calling `setDoc(doc(db, "users", newAuthUid), ...)`.
- **Retest Status:** Open.

FEASIFY-BUG-007: Unauthenticated Access to Gemini AI Analysis Endpoints

- **Module:** Backend Companion / AI Engine
- **Severity: High** | **Priority: P2**
- **Affected File:** `feasify-backend/index.js` (`/api/analyze` and `/api/analyze-proposal`)
- **Preconditions:** Backend running.
- **Steps to Reproduce:** Send `POST /api/analyze` with financial payload without `Authorization` header.
- **Expected Result:** HTTP `401 Unauthorized` .
- **Actual Result:** HTTP `200 OK` ; Gemini API request dispatched.
- **Root Cause:** `verifyToken` middleware omitted on AI routes.
- **Recommended Solution:** Attach `verifyToken` middleware to ensure only enrolled students/advisers consume AI credits.
- **Retest Status:** Open.

FEASIFY-BUG-008: Hardcoded Chairperson Email Privilege Escalation

- **Module:** Access Control / Route Protection
- **Severity: High** | **Priority: P2**
- **Affected Files:** `ProtectedRoute.tsx` (line 31), `Auth.tsx` (line 151), `feasify-backend/index.js` (line 97)
- **Preconditions:** User signs up with email `chairperson@gmail.com` .
- **Steps to Reproduce:** Log in using `chairperson@gmail.com` .
- **Expected Result:** Role loaded from database; unauthorized if not explicitly appointed.
- **Actual Result:** User automatically granted full Chairperson access across frontend and backend.
- **Root Cause:** Hardcoded email check `user.email?.toLowerCase() === "chairperson@gmail.com"` .
- **Recommended Solution:** Remove email string check; enforce role verification from Firestore document or custom claims.
- **Retest Status:** Open.

FEASIFY-BUG-009: Null Dereference Risk in Adviser Dashboard Financial View

- **Module:** Adviser Dashboard
- **Severity: Medium** | **Priority: P3**
- **Affected File:** `Feasify/src/AdviserDashboard.tsx` (lines 1479-1488)
- **Preconditions:** Group has financial data but `activeProposal` is null.
- **Steps to Reproduce:** Open financial review tab for a group that has no active approved proposal.
- **Expected Result:** UI renders empty state notice: "No active proposal selected".
- **Actual Result:** Uncaught `TypeError: Cannot read properties of null (reading 'products')` or `(reading 'businessName')` .

- **Root Cause:** `activeProposal.products` and `activeProposal.businessName` accessed without optional chaining `activeProposal?.products`.
- **Recommended Solution:** Add optional chaining: `activeProposal?.products || []` and guard clause.
- **Retest Status:** Open.

FEASIFY-BUG-010: Hardcoded EmailJS Public Key in Client Code

- **Module:** Chairperson Administration
- **Severity: Medium** | **Priority: P3**
- **Affected File:** `Feasify/src/ChairpersonModule.tsx` (lines 285, 509)
- **Preconditions:** Source inspection.
- **Steps to Reproduce:** Search for `"Iw4MKLYpB4TPgpXLn"`.
- **Expected Result:** Credentials loaded from environment configuration.
- **Actual Result:** Key hardcoded as string literal in source file.
- **Root Cause:** Developer embedded key directly into component call.
- **Recommended Solution:** Extract to `import.meta.env.VITE_EMAILJS_PUBLIC_KEY`.
- **Retest Status:** Open.

FEASIFY-BUG-011: Wildcard Cross-Origin Resource Sharing (CORS) Configuration

- **Module:** Companion Backend
- **Severity: Medium** | **Priority: P3**
- **Affected File:** `feasify-backend/index.js` (lines 133, 152)
- **Preconditions:** Backend running.
- **Steps to Reproduce:** Send request with arbitrary `Origin: http://malicious-site.example`.
- **Expected Result:** Backend restricts CORS to authorized frontend domain.
- **Actual Result:** Response includes `Access-Control-Allow-Origin: *`.
- **Root Cause:** `app.use(cors())` and `io = new Server(server, { cors: { origin: "*" } })` default to wildcard.
- **Recommended Solution:** Restrict origin to `process.env.FRONTEND_URL || "http://localhost:5173"`.
- **Retest Status:** Open.

FEASIFY-BUG-012: Large Monolithic Production Bundle Size

- **Module:** Frontend Build / Performance
- **Severity: Low** | **Priority: P4**
- **Affected File:** `Feasify/src/App.tsx` / `dist/assets/index-*.js`
- **Preconditions:** Run `npm run build --prefix Feasify`.
- **Steps to Reproduce:** Inspect Vite build terminal output.

- **Expected Result:** Chunks under 500 kB.
- **Actual Result:** Main chunk is **3,362.13 kB** uncompressed.
- **Root Cause:** All modules imported synchronously in `App.tsx`.
- **Recommended Solution:** Implement `React.lazy` on major route components.
- **Retest Status:** Open.

FEASIFY-BUG-013: ContributorItem Interface Property Mismatch

- **Module:** Financial Utilities
- **Severity: Low** | **Priority: P4**
- **Affected File:** `Feasify/src/utils/cabbageBenchmark.ts` (lines 47-55)
- **Preconditions:** Run `tsc --noEmit`.
- **Steps to Reproduce:** Compile TypeScript files.
- **Expected Result:** TypeScript compiles with zero errors.
- **Actual Result:** `error TS2353: Object literal may only specify known properties, and 'role' does not exist in type 'ContributorItem'`.
- **Root Cause:** Benchmark template contains `role` property not declared in `ContributorItem` type in `productCosting.ts`.
- **Recommended Solution:** Add `role?: string` to `ContributorItem` interface.
- **Retest Status:** Open.
